# R1:  Static master-worker scheduling and correctness verification

For static worker scheduling, the master worker initializes the routine by chunking up the work and sending that work to the workers through MPI_Send.  Each worker waits for a signal from the Master as either a stop signal, or a chunk they are to work on.  After a worker receives a chunk to work on, they count the primes in that chunk, then they send their partial prime counts through MPI_SEND back to the master, and waits to hear back from master (either another chunk, or a stop signal).  After all the chunks have been evaluated, the master sends a stop signal to all the workers (TAG_STOP).  Then, the master collects all the prime counts using MPI_Recv.  Effectively, the master loops through the workers in three distinct loops:
Loop 1 - send chunks to workers
Loop 2 - collect results
Loop 3 -  stops all workers

Below is the terminal output showing evidence that the distributed result matches the sequential result for a small case (N = 200,000 and CHUNK size = 50,000)

```
Shell
$ mpirun -np 4 ./r1 200000 50000
Master: send [2..50001] to worker 1
Master: send [50002..100001] to worker 2
Master: send [100002..150001] to worker 3
Master: send [150002..200000] to worker 1
worker 1: processed [2..50001] -> 5133 primes
worker 2: processed [50002..100001] -> 4459 primes
worker 3: processed [100002..150001] -> 4257 primes
worker 1: processed [150002..200000] -> 4135 primes
Parallel Result: Total primes in [2, 200000] = 17984
Sequential Verification: Total primes = 17984
VERIFICATION PASSED!
R1 Execution Time: 0.008986 seconds
```

# R2: Dynamic master-worker scheduling and correctness verification

For Dynamic scheduling, the communication is handled quite differently.  Instead of the master pushing the work in a deterministic order over loops, the master is waiting to receive any signals from idle workers (when initialized, all are idle).  If the workers request work, the master keeps track of the last chunk sent, and sends the next chunk directly to the worker requesting it.  This repeats until all work has been assigned.  Once all work has been assigned, subsequent worker requests are handled with a stop work signal (TAG_STOP), and the workers are terminated.

Below is the terminal output showing evidence that the distributed result matches the sequential result for a small case (N = 200,000 and CHUNK size = 50,000):

```
Shell
$ mpirun -np 4 ./r2 200000 50000
Master: send [2..50001] to worker 2
Master: send [50002..100001] to worker 1
Master: send [100002..150001] to worker 3
Master: send [150002..200000] to worker 2
Parallel Result: Total primes in [2, 200000] = 17984
Sequential Verification: Total primes = 17984
VERIFICATION PASSED!
Elapsed Parallel Time: 0.009091 seconds
```

# R2: Static vs. Dynamic Comparison:

For the performance comparison, I used N=100,000,000 and CHUNK = 50,000 with 1 master and 3 workers. I turned off all in-process I/o, and ran both the dynamic and static models 3 times. Results are summarized in Table 1 below.

Table 1: Static and Dynamic worker comparison

| | Trial 1 (s) | Trial 2 (s) | Trial 3 (s) | Average |
|---|---|---|---|---|
| Static Model (r1) | 11.631177 | 11.628168 | 11.623601 | 11.627649 |
| Dynamic Model (r2) | 11.560098 | 11.320715 | 11.31862 | 11.399811 |

Under these conditions, the dynamic model is slightly faster. This is because load balancing outweighs the additional communication overhead caused by the extra message requests in the dynamic model. This load imbalance is due to the fact that workers on the top end of the static model, (3 workers for example), would have to check up to $\sqrt{100,000,000} = 10,000$ divisors. Worker 1 would sit idle waiting. In the dynamic model, workers 1 and 2 pull chunks continuously, keeping all processes busy until work is complete.

# R3: Rushmore Scaling Study

Table 2. Small Problem Size (N=10,000,000, CHUNK = 50,000, Total Primes = 664,579)

| Processes (P) | Node Placement | Trial 1 (s) | Trial 2 (s) | Trial 3 (s) | Avg Time T | Relative Speedup S(P) | Relative Efficiency E(P) |
|---|---|---|---|---|---|---|---|
| 4 | 1 per vm | 0.783746 | 0.815534 | 0.700323 | 0.766534 | 1.000 | 1.000 |
| 8 | 2 per vm | 0.903956 | 2.020551 | 1.085788 | 1.336765 | 0.573 | 0.287 |
| 16 | 4 per vm | 2.820676 | 2.624135 | 2.595979 | 2.680263 | 0.286 | 0.071 |

## Small problem size analysis:

The trend illustrated in table 2 shows that execution time increases as process count grows. The result is poor efficiency. This may be because the total computational work is too small (less than 1 second on 4 processes). The dynamic schedule message-passing overhead (request-work/assign-work pairs) and MPI initialization synchronization completely dominate the total runtime. Adding processes adds network traffic without providing enough parallel work per chunk to offset the communication costs.

Table 3. Medium Problem Size (N=50,000,000 with CHUNK = 50,000, Total Primes = 3,001,134)

| Processes (P) | Node Placement | Trial 1 (s) | Trial 2 (s) | Trial 3 (s) | Avg Time T | Relative Speedup S(P) | Relative Efficiency E(P) |
|---|---|---|---|---|---|---|---|
| 4 | 1 per vm | 7.929890 | 9.815485 | 10.253831 | 9.333069 | 1.000 | 1.000 |
| 8 | 2 per vm | 5.684853 | 6.078753 | 7.318140 | 6.360582 | 1.467 | 0.734 |
| 16 | 4 per vm | 7.935868 | 6.728193 | 8.121775 | 7.595279 | 1.229 | 0.307 |

## Medium problem size analysis:

I observed a relative speedup from 4 processes to 8, but speed dropped when increasing to 16 processes. Moving from 4 processes to 8 processes, speed increases because the problem size is sufficiently large for parallel computation to outweigh the additional communication costs. When moving up to 16 total processes, 4 per VM, I observed a slow down when compared to 4 and 8 processes. This may be due to a congested network message queue, or some of the communication bottleneck on rank 0 (the master worker).

Table 4. Larch Problem Size (N=100,000,000 with CHUNK = 50,000, Total Primes = 5,761,455)

| Processes (P) | Node Placement | Trial 1 (s) | Trial 2 (s) | Trial 3 (s) | Avg Time T | Relative Speedup S(P) | Relative Efficiency E(P) |
|---|---|---|---|---|---|---|---|
| 4 | 1 per vm | 41.780155 | 44.206096 | 45.603118 | 43.863123 | 1.000 | 1.000 |
| 8 | 2 per vm | 66.798108 | 63.869490 | 69.422560 | 66.696719 | 0.658 | 0.329 |
| 16 | 4 per vm | 42.260192 | 62.595734 | 52.300096 | 52.385341 | 0.837 | 0.209 |

## Large problem size analysis:

The 4-process configuration outperforms both the 8 processes and the 16 processes configuration. With 4 processes (1 process per virtual machine), each process has dedicated hardware resources and minimal inter-process contention. The lower performance observed with 8 and 16 processes may be due to larger processes consuming larger amounts of resources, and the master worker attempting to service dynamic requests from 15 active processes.